Assignment 01: Walking while carrying a tray with TSID

Pietro Noah Crestaz* - pietronoah.crestaz@unitn.it

October 2026

1 Description

This assignment focuses on Task Space Inverse Dynamics (TSID) for biped walking. The humanoid robot Romeo has to walk while carrying a tray with its right hand, like a waiter. The goals of this assignment are:

- understanding how contacts are created and removed in TSID, and the role of double support phases in walking;
- adding a new task to a TSID controller, choosing which of its components matter, and understanding the difference between weighted tasks and hard constraints in TSID.

2 Submission procedure

You are encouraged to work on the assignments in groups of 2 people. **If you have a good reason to work alone, then you can do it, but this has to be previously validated by one of the instructors**. Groups of more than 2 people are not allowed. The mark of each assignment contributes to 10% of your final mark for the class (i.e. 3 points out of 30).

When you are done with the assignment, please submit a single compressed file (e.g., zip). **The file name should contain the ID number of the group members**, and it must contain:

- A pdf file with the answers to the questions, the **names and ID number** of the group members; you are encouraged to include plots and/or numerical values obtained through simulations to support your answers. **This pdf does not need to be long. One or two pages of text should be enough to answer the questions. You can then add other pages for plots and tables.**
- The complete folder containing all the python code that you have developed.

If you are working in a group (i.e., 2 people) only one of you has to submit.

Submitting the pdf file without the code is not allowed and would result in zero points. Your code should be consistent with your answers (i.e. it should be possible to produce the results that motivated your answers using the code that you submitted). If your code does not even run, then your mark will be zero, so make sure to submit a correct code.

*Optimization and Learning for Robot Control, Industrial Engineering Department, University of Trento.

3 Background

Reference trajectories. The robot walks six steps following reference trajectories of the CoM, of the feet and of the CoP (Center of Pressure), which are provided in two files. In `romeo_walking_ds.npz` (the default) single support phases, with only one foot on the ground, alternate with double support phases of 0.2 s, with both feet on the ground, in which the weight is transferred from one foot to the other. In `romeo_walking_no_ds.npz` there are no double support phases: the swing foot touches the ground at the very instant in which the other foot is lifted. The file is selected with `DATA_FILE_TSID` in `romeo_conf.py`. The files also contain, for every time step i, a label `contact_phase[i]`, which is `'double'` (both feet on the ground), `'right'` (only the right foot on the ground) or `'left'` (only the left foot on the ground).

Removing a contact in TSID. A contact is removed with `TsidBiped.remove_contact_RF(transition_time)` (similarly for the left foot). If `transition_time` is zero the contact is removed immediately. Otherwise, TSID keeps the contact active for `transition_time` seconds while decreasing linearly the maximum normal force that the foot can exert, from its current value to zero, and only then removes the contact. Contacts are added with `add_contact_RF()` and `add_contact_LF()`. The function `compute_contact_events()` in `biped_walking.py` uses the contact sequence of the reference to decide when contacts are added (at touchdown) and when their removal starts: `transition_time` seconds *before* lift-off, so that the force ramp is over exactly when the swing foot starts moving.

Tray task. The tray is used only in the second part of the assignment, and is enabled by setting `tray = True` in `romeo_conf.py`. It is carried by the right hand, whose frame is `hand_frame_name`. A `tsid.TaskSE3Equality` on this frame controls its position and orientation. Its *mask* (a vector of 6 elements, 3 for the position and 3 for the orientation) selects which components of the task are enforced, while the method `useLocalFrame()` selects whether the task (and hence its mask) is expressed in the local frame of the hand or in a frame aligned with the world. The mass of the tray and of what it carries is set by `payload_mass` (0.5 kg by default, i.e. the empty tray): it is added to the model of the hand, so that TSID is aware of it. The *tilt* of the tray is defined as the angle between its current vertical axis and its initial one: a tilted tray spills its glasses. The tray and two glasses are drawn in the viewer, rigidly attached to the hand.

4 Tests

The template code for the assignment is located in our shared folder:

`lab/A1_template`

In this folder you will find:

- `romeo_conf.py`: the parameters of the robot and of TSID, and the reference file to use;
- `romeo_walking_ds.npz`, `romeo_walking_no_ds.npz`: the reference trajectories described above;
- `tray_sinusoid.py`: the robot stands and moves the tray along a sinusoidal trajectory (question 2);
- `tsid_biped.py`: the TSID controller of the biped (class `TsidBiped`), extended with the tray task and the payload;
- `biped_walking.py`: simulates the walk of the humanoid Romeo, prints some performance indicators (CoM tracking error, maximum rate of change of the normal forces and of the joint

torques, maximum tilt of the tray, maximum hand position error, range of heights of the hand, and the joint whose torque gets closest to its limit) and plots the results;

- `compare_payloads.py`: simulates the walk with an empty tray and with a heavy tray, and shows them together in the viewer (question 5).

Please do not change the parameters in `romeo_conf.py` unless you are explicitly asked to do so. Whenever you change a parameter to answer a question, restore its default value before moving to the next question, unless stated otherwise. You can set `viewer = None` in `romeo_conf.py` to run the simulations faster without visualisation.

4.1 Part 1: Walking with double support

In this part the robot does not carry the tray (`tray = False`, the default).

1. Run `biped_walking.py` with the default reference file, and then with `romeo_walking_no_ds.npz` (no double support). Report the plots of the normal forces of the feet (the plot of the CoM also shows the reference CoP) and the maximum rate of change of the normal forces and of the joint torques in the two cases. Describe how the weight of the robot is transferred from one foot to the other, relating it to the reference CoP, and explain the difference between the two cases. Why could the case without double support be a problem on a real robot, even though it works in this simulation?

   Then go back to the default reference file and set `transition_time = 0.1`: the robot falls down. Use the plots of the normal forces and of the *maximum* normal forces of the feet to find out why, and fix the problem by modifying only the methods `add_contact_RF()` and `add_contact_LF()` of `TsidBiped`. Explain your fix. Once fixed, does the force ramp make a large difference with respect to `transition_time = 0` with this gait? Why?

4.2 Part 2: Carrying a tray

In this part set `tray = True` in `romeo_conf.py` (the scripts `tray_sinusoid.py` and `compare_payloads.py` do it by themselves). When walking, use the default reference file and keep `transition_time = 0.1` with the fix of question 1.

2. Complete the tray task in `TsidBiped.__init__()` (gains, mask, frame, weight and priority level, as described in the code) and the function `sinusoid_reference()` in `tray_sinusoid.py`, which computes the position, velocity and acceleration of a sinusoidal reference for the hand. Run `tray_sinusoid.py`: the robot stands still and moves the tray up and down (amplitude 5 cm) at the frequency `FREQUENCY`, while keeping it horizontal. Report the amplitude ratio and the phase lag between the hand and its reference for `FREQUENCY` equal to 0.5 Hz and 2 Hz, with and without feedforward (`FEEDFORWARD = False` sets the velocity and acceleration references to zero).

   Without feedforward, why does the hand lag behind its reference and move less than it, and why does this get worse at 2 Hz? (Hint: think about which terms of the desired acceleration of the task are left.) With feedforward the tracking is much better, but it is not perfect, especially at 2 Hz: why? (Hint: look at the CoM.)

3. Compute the reference of the tray task in `biped_walking.py`: the hand must keep its initial orientation and height, while moving horizontally (x and y) together with the reference of the CoM, with consistent velocity and acceleration references. Compare the walk without the tray task (`w_hand = 0`) and with it (`w_hand = 1`): report the tilt of the tray, the hand tracking and the CoM tracking. Why does the CoM tracking get worse when the tray task is active, even though the arm is light?

4. To avoid spilling, a waiter only needs to keep the tray horizontal and at a constant height: its horizontal position and its rotation about the vertical axis do not matter. Test the tray task controlling only the position of the hand (mask `[1,1,1,0,0,0]`) and only its orientation (mask `[0,0,0,1,1,1]`), then choose the mask that implements exactly the requirement of the waiter and test it with `hand_local_frame` set to `True` and to `False`. Report the tilt of the tray, the height range of the hand and the CoM tracking error in all cases, and explain the results.

5. **Heavy trays.** Use again the full tray task (mask `[1,1,1,1,1,1]`, local frame). First, run `compare_payloads.py`: it simulates the walk with a 5 kg tray (with raised torque limits, since Romeo could not carry it otherwise) and shows it together with the baseline walk (empty tray), drawn as a blue transparent robot, with markers for the total CoM and the pelvis. Describe how the robot changes its posture to carry the heavy tray: how much and in which direction does the pelvis move, and what happens to the total CoM? Explain why, based on the tasks of the controller.

   Then, with the nominal torque limits, put two full glasses on the tray (`payload_mass = 2`) and compare the walk with `w_hand = 100` and with the tray task as a hard constraint (`hand_priority = 0`). With the hard constraint the QP becomes infeasible: which other constraint of the problem is in conflict with the tray task? Support your answer with the plot of the normalised joint torques. What does the weighted task sacrifice to keep the robot walking? Based on these results, discuss when a task should be a hard constraint and when it should be a weighted cost.